Unitree Week 35

Presentation by Mario Ortiz

This Week

- R1 Startup
- Reinforcement Learning design for Upper Body Motion (Shoulder movement)

Other RL ideas

- Since we don’t want to actually start locomotion tasks for the R1, I will work on developing training models for simpler tasks (primarily moving the arms).
  - A simple training could be to have the R1 move its arms up and down repeatedly (controlled by a controller button to stop).

Note on RL/IL

- Since the research we want to work on is more focused on finding methods to teach the Unitree to analyze environmental information, I will focus less on specific algorithm design for the Unitree and focus on obtaining trained models for the R1.
- From online research, there are a significant amount of RL tasks for the Unitree G1 rather than the R1. Therefore, converting the G1 tasks so they align with the R1 could be a method to obtaining more trained models.
-

Simple Upper Body RL Task

- Instead of deploying the locomotion task, I could deploy a task that doesn't require the R1 to do any movements. In this case, I could deploy a task onto the R1 that just raises its arms when desired.
- A control mechanism could be implemented which causes:
  - 0.0 → arms down
  - 0.3 → slightly raises
  - ….
  - 1.0 → arms fully raised.

Simple Upper Body RL Task

- We can start with the “shoulder_pitch_joint”s of the R1.
- Information states that the shoulder pitch joint has a “home configuration” of approximately 0.35 rad. (increasing/decreasing this results in controlling how we want the arm to move).

Simple Upper Body RL Task: Observations

The information needed for the R1 to initiate the task:

- Current left/right shoulder angle
- Current left/right shoulder velocity
- Commanded shoulder position
- Previous action
- Projected Gravity
- Base Angular Velocity

Simple Upper Body RL Task: Reward

The reward is also less complex than the locomotion task:

- Compute the error for each shoulder
- Reward function relates to error of both shoulders.
- Now reward quantity is provided based on how high error is:
  - Arms very close to requested position: reward ~= 1
  - Arms somewhat wrong: smaller
  - Arms completely wrong: reward ~= 0
- Supporting penalties also added:
  - Moves violently
  - Exceeds Safe joint range
  - Collide arms with torso/head

Simple Upper Body RL Task: Termination

Termination occurs if:

- Joint exceeds safety bound
- Self-collision occurs
- Unexpected torso orientation becomes excessive

Simple Upper Body RL Task: Termination

Files made:

- __init__.py
- hanger_arms_env_cfg.py
- mdp
  - __init__.py
  - commands.py
  - rewards.py
-

- config/
  - __init__.py
  - r1/
    - __init__.py
    - env_cfgs.py
    - R1_cfg.py
- r1/
  - __init__.py
  - joint_sweep.py
  - manual_play.py

Hanger Simulation Model

- The original R1 model was copied into “r1_hanger.xml”, and the floating based was replaced with a fixed pelvis/base to simulate the robot hanging (similar to how the physical robot is currently hanged on the bracket)

Shoulder Command

- The initial test is with only the shoulders.
- The task command inputs c = [c_L, c_R]
  - Left and right shoulder-pitch offset.
- Tracking command ranges: -0.25 <= c_L, c_R <= +0.25 rad
  - Desired joint position is then q_target = q_default + c
- These commands were randomly regenerated approximately every 2-4 seconds during training.
  - This ensured the network did not learn one fixed animation and learned to respond to different requested shoulder positions.

Action Control

- MJLab's joint-position control: q_command = q_default + s*a
  - a = RL network output
  - s = R1 joint-specific action scale
  - q_defalt = R1 nominal joint position
- We have the policy operating 0.005 s physics step * 4 = 0.020s (50 Hz policy frequency)

Reward Function

- The policy was trained to move the shoulder-pitch joints toward their command positions while avoiding unnecessary motion.
- Rewards included:
  - arm_tracking
  - non_task_posture
  - joint_pos_limits
  - action_rate_12
  - joint_acc_12
  - Self_collisions

Custom Validation Tool;

- This was made to automatically test every policy output using:
  - [0, -0.5, -1, -0.5, 0, +0.5, +1, +0.5, 0] for each of the 24 action dimensions.
  - This verified action ordering, correct actuator assignment, joint direction, action scaling, and default-pose return behavior.
-

Training

- A short validation run was performed:
  - 256 parallel R1 environments
  - 100 PPO iterations
  - 24 simulation steps
- Hence 256 * 24 * 100 = 614,400 simulation steps
- Time to train: 1 minute 18 seconds

- Results:
  - Mean reward: 12.09
  - Arm tracking reward: 1.5975
  - Non-task posture penalty: -0.0222
  - Joint-limit penalty: -0.0004
  - Action-rate: -0.4840
  - Acceleration penalty: -0.0634
  - Self-collision penalty: -0.0051
  -

Visualisation

- On Viser, we can provide the joint commands we want the R1 to do and then view the R1 initiating the tasks.
- There is also information regarding error in the distance of the movements.



References

- https://github.com/unitreerobotics/unitree_rl_mjlab
- https://github.com/unitreerobotics/unitree_rl_mjlab/blob/main/src/assets/robots/unitree_r1/r1_constants.py
- https://github.com/unitreerobotics/unitree_rl_mjlab/blob/main/scripts/play.py